

1. Do the following problems from this [book](#) : 5.7, 5.8

5.7

- a) I make a design matrix 'A' to include the polynomial functions and vary the degree from 1 to 10.

```
function A = design_poly_matrix(t,degree)
A = ones(length(t),1);
A = [A t];
for i = 1:degree - 1
    A = [A A(:,i+1) .* t];
end
end
```

Then, I run the experiment which solves least square problems via matlab backslash using the following script.

```
% generate data points t and observations y = erf(t)
k = (1:11)';
t = (k - 1) / 10;
y = erf(t);
err = [];
for degree = 1:11
    % construct a polynomial design matrix of degree
    1:11
    A = design_poly_matrix(t,degree);
    x = A \ y; % solve least square
    err = [err;norm(A*x - y,Inf)];
end
```

The vector 'err' is the maximum error measured by infinity norm. (The question specifies the maximum error.)

	1	2	3	4	5	6	7	8	9	10	11
err	0.0635										
1	0.0635										
2	0.0063										
3	0.0012										
4	4.6596e-05										
5	2.4099e-05										
6	6.9575e-07										
7	1.9394e-07										
8	7.1888e-09										
9	7.2529e-10										
10	2.2204e-16										
11											
12											
13											
14											

The maximum error is monotonically decreasing for higher degree polynomials. Polynomial degree 10 has the minimum error of around 2e-16. Degree 4 already gives a small error of around 4e-5.

- b) Again, I make a design matrix 'A' but include only the 'odd' power of the polynomial functions and vary the maximum degree from 1 to 19. ($2n-1$ where n is 1 to 10.)

```
function A = design_odd_poly_matrix(t,degree)
A = ones(length(t),1);
A = [A t];
for i = 1:degree - 1
    A = [A (A(:,i+1) .* t).^t];
end
end
```

Then, I run the experiment which solves least square problems via matlab backslash using the following script.

```
% generate data points t and observations y = erf(t)
k = (1:11)';
t = (k - 1) / 10;
y = erf(t);
err = [];
for degree = 1:10
    % construct an 'odd' power of polynomial design
    matrix of degree 1:11
    A = design_odd_poly_matrix(t,degree);
    x = A \ y; % solve least square
    err = [err;norm(A*x-y,Inf)];
end
```

The vector 'err' is the maximum error measured by infinity norm. (The question specifies the maximum error.)

	1	2	3	4	5	6	7	8	9	10	11
1	0.0635										
2	0.0046										
3	3.2001e-04										
4	1.3734e-05										
5	5.2161e-07										
6	1.3364e-08										
7	2.4750e-10										
8	2.8313e-12										
9	1.9873e-14										
10	1.1102e-16										
11											
12											
13											
14											

The maximum error is monotonically decreasing for higher degree polynomials. Odd polynomial degree 19 has the minimum error of around 1e-16. Odd polynomial degree 5 already gives a small error of 3e-4.

- c) Again, I make a design matrix 'A' according to the equation specified in the question.

```
function A = design_bound_matrix(t,degree)
A = ones(length(t),1);
e_t = exp(-(t).^2);
A = [A e_t];
z = ones(length(t),1) ./ (1.0 + t);
for i = 1:degree - 1
    A = [A (A(:, i + 1) .* z)];
end
end
```

Then, I run the experiment which solves least square problems via matlab backslash using the following script.

```
% generate data points t and observations y = erf(t)
k = (1:11)';
t = (k - 1) / 10;
y = erf(t);
err = [];
for degree = 1:10
    % construct a bound polynomial function design matrix
    % of degree 1:11
    A = design_bound_matrix(t,degree);
    x = A \ y; % solve least square
    err = [err;norm(A*x-y,Inf)];
end
```

The vector 'err' is the maximum error measured by infinity norm. (The question specifies the maximum error.)

	1	2	3	4	5	6	7	8	9	10	11
1	0.1661										
2	8.4703e-04										
3	3.0712e-04										
4	2.6711e-05										
5	8.4872e-07										
6	7.2666e-08										
7	8.7266e-09										
8	4.7111e-10										
9	6.5714e-12										
10	6.6613e-16										
11											
12											
13											
14											

The maximum error is monotonically decreasing for higher degree polynomials. Bound polynomial degree 10 has the minimum error of around 6e-16. However, bound polynomial degree 2 already gives a small error of 8e-4. This means bound polynomial can fit the data better than a normal polynomial or odd polynomial function.

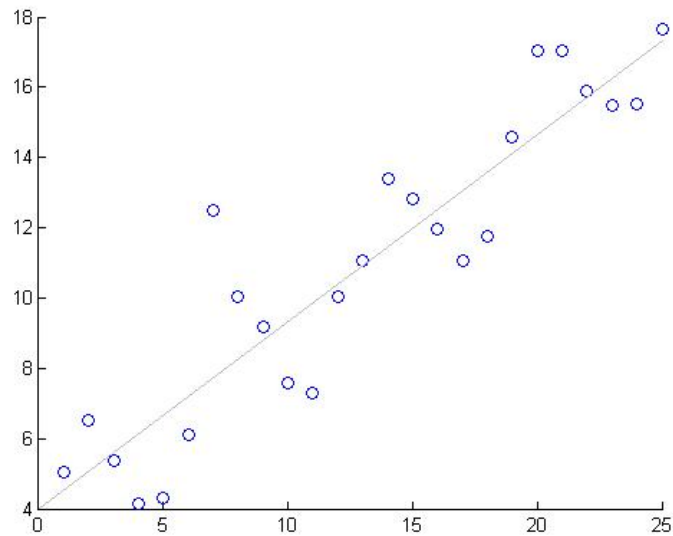
a), b), c) shows the results of fitting different functions into the data and evaluate as a fit to an outlier point.

5.8

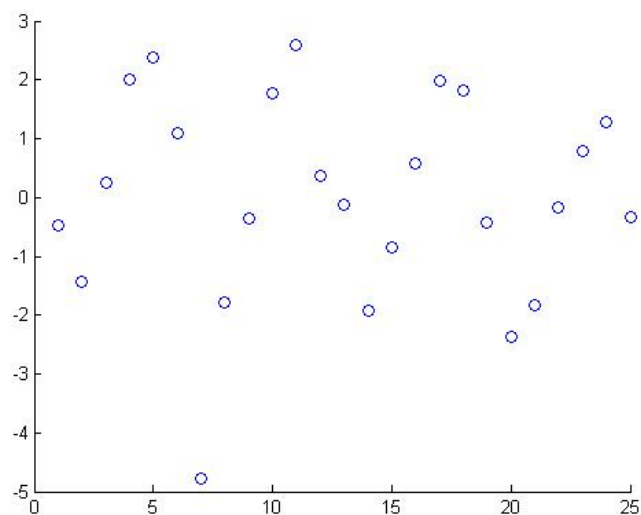
a) After generating the data points as described in the question, I fit the data points with a straight line by using the following code.

```
% make a design matrix A
A = ones(length(t),1);
A = [A t];
% fit the data
x = A \ y;
% plot
scatter(t,y);
hold on;
lsline;
```

The resulting plot is as follows,



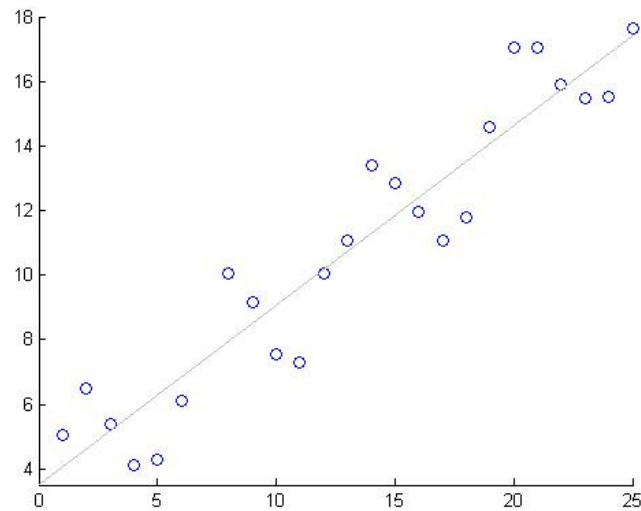
Then, I plot the residual error and observe that the 7th data point is an outlier since it has the highest residual error.



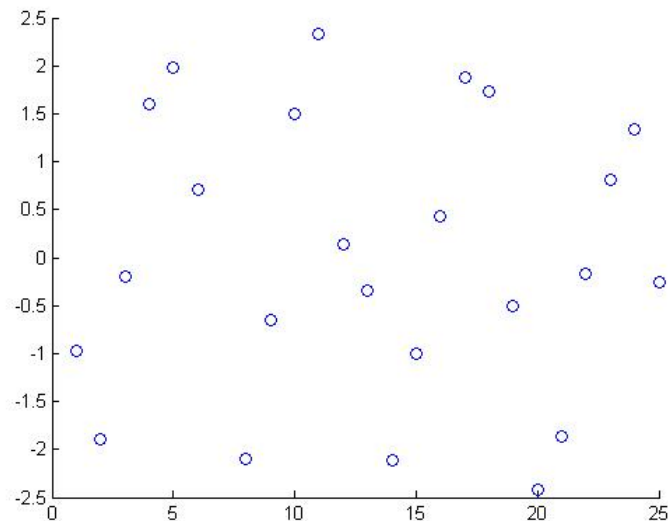
b) I discard the 7th data point then rerun the experiment.

```
% discard the 7th data point  
t(7) = [];  
y(7) = [];
```

The resulting plot is as follows,



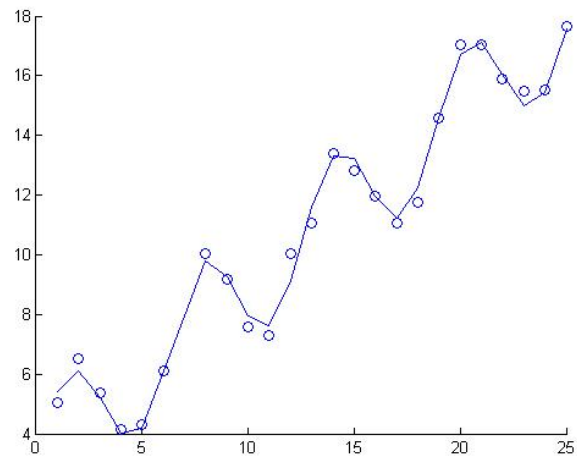
Then, I plot the residual error and observe that the residual errors lie in the range 2.5 to -2.5. The errors also go up and down in some kind of interval.



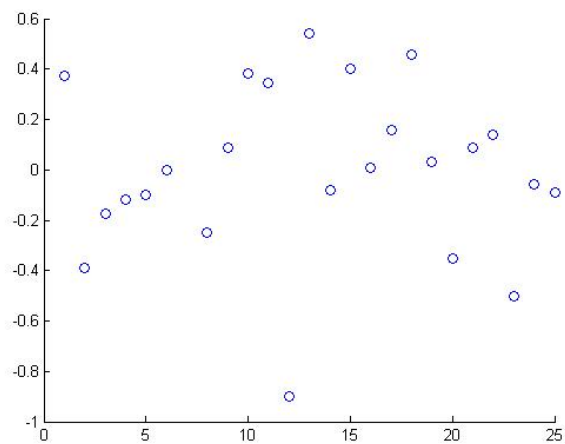
- c) I change the design matrix to include the term $\sin(t)$ then I rerun the experiment on the data set without the outlier.

```
% make a design matrix A
A = ones(length(t),1);
A = [A t sin(t)];
```

The resulting plot is as follows,



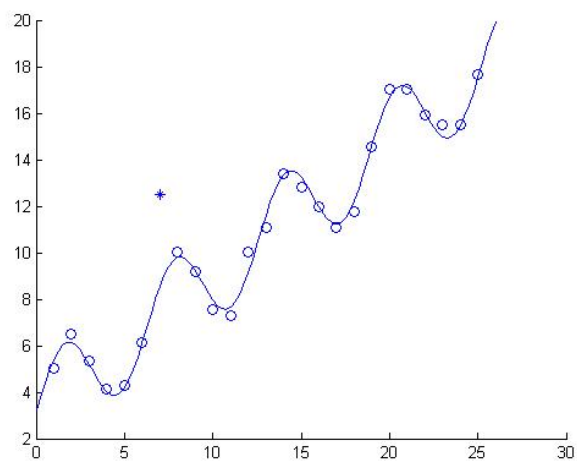
Then, I plot the residual error and observe that the residual errors lie in the more narrow range 0.6 to -0.9.



d) I make a finer grid by redesign matrix G . Then, I compute the fit by $G \cdot x$.

```
% make a finer design matrix G
g = 0:0.25:26;
g = g';
G = ones(length(g),1);
G = [G g sin(g)];
```

Then, I plot the fitted curve.



The fitted curve seems to fit the data well when excluding the outlier.

2. Do Challenge 5.8 in your textbook. Also do the problem via Householder reflection. (This is a problem to do by hand).

Compute the qr decomposition of the matrix $A = \begin{bmatrix} 3 & 3 \\ 3 & 1 \end{bmatrix}$.

- a) Givens rotation

(From algorithm 5.1 in the textbook.)

We need to zero out (2, 1) so the $G(2, 1)$ is $\begin{bmatrix} c & s \\ s & -c \end{bmatrix}$.

Since $z_1 = 3$ and $z_2 = 3$, $r = 3\sqrt{2}$, $c = z_1 / r = 1 / \sqrt{2}$.

Likewise, $s = z_2 / r = 1 / \sqrt{2} \sim 0.7071$.

We get the matrix $G(2, 1) = \begin{bmatrix} 0.7071 & 0.7071 \\ 0.7071 & -0.7071 \end{bmatrix}$.

Then, $A = G(2, 1) * A = \begin{bmatrix} 4.2426 & 2.8284 \\ 0 & 1.4142 \end{bmatrix}$.

Since A is already upper triangular, $R = \begin{bmatrix} 4.2426 & 2.8284 \\ 0 & 1.4142 \end{bmatrix}$. #

And we get $Q = G(2, 1)' = \begin{bmatrix} 0.7071 & 0.7071 \\ 0.7071 & -0.7071 \end{bmatrix}$. #

- b) Gram-Schmidt orthogonalization

(From algorithm 5.2 in the textbook.)

$r_{11} = \text{norm of } a_1 = \text{norm}(\begin{bmatrix} 3 \\ 3 \end{bmatrix}) = 4.2426$.

$q_1 = a_1 / r_{11} = \begin{bmatrix} 3 \\ 3 \end{bmatrix} / 4.2426 = \begin{bmatrix} 0.7071 \\ 0.7071 \end{bmatrix}$.

Let $q_2 = a_2 = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$.

$r_{12} = q_1' * q_2 = 2.8284$.

$q_2 = q_2 - r_{12} * q_1 = \begin{bmatrix} 3 \\ 1 \end{bmatrix} - 2.8284 * \begin{bmatrix} 0.7071 \\ 0.7071 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$.

$r_{22} = \text{norm}(q_2) = 1.4142$.

$q_2 = q_2 / r_{22} = \begin{bmatrix} 0.7071 \\ -0.7071 \end{bmatrix}$.

Hence, $R = \begin{bmatrix} 4.2426 & 2.8284 \\ 0 & 1.4142 \end{bmatrix}$ and $Q = \begin{bmatrix} 0.7071 & 0.7071 \\ 0.7071 & -0.7071 \end{bmatrix}$. #

- c) Householder reflection

Find a vector u from the first column of A denoted as a_1 .

Denotes e_1 as $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$.

$u = a_1 - \text{norm}(a_1)e_1 = \begin{bmatrix} 3 \\ 3 \end{bmatrix} - 4.2426 * \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -1.2426 \\ 3 \end{bmatrix}$

Then, normalize $u = u / \text{norm}(u) = \begin{bmatrix} -0.3827 \\ 0.9239 \end{bmatrix}$.

Construct $H = I - 2 * u * u' = \begin{bmatrix} 0.7071 & 0.7071 \\ 0.7071 & -0.7071 \end{bmatrix}$.

$A = H * A = \begin{bmatrix} 4.2426 & 2.8284 \\ 0 & 1.4142 \end{bmatrix}$.

Since A is upper triangular R is A which means $R = \begin{bmatrix} 4.2426 & 2.8284 \\ 0 & 1.4142 \end{bmatrix}$. #

And $Q = H' = \begin{bmatrix} 0.7071 & 0.7071 \\ 0.7071 & -0.7071 \end{bmatrix}$. #

3. Following Pointer 5.8 of the textbook. Create a random matrix A of size 4×4 , and compute the best rank 1, rank 2, and rank 3 approximations, A_r to that matrix. Compute the Frobenius norm of $\|A - A_r\|, r=1,2,3$

Since SVD expresses the matrix A as the p -terms sum of rank-1 matrices which is the best rank- p approximation of that matrix A .

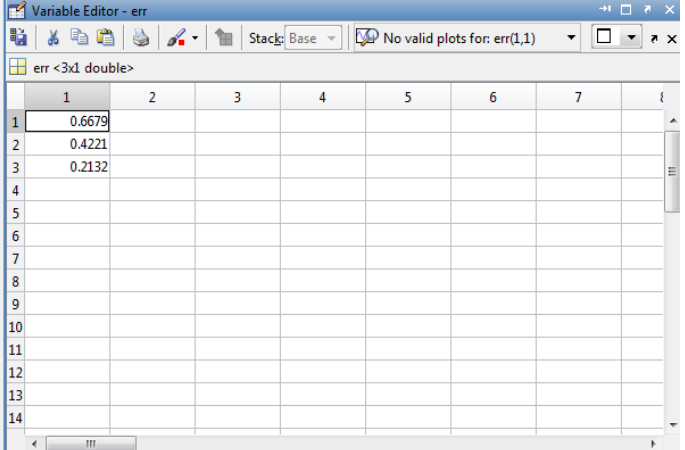
I created a random 4 by 4 matrix A and perform SVD on A . For rank- p approximation, I used p columns of U , diagonal entries of S and rows of V transpose where p is from 1 to 3.

```
A = rand(4,4); % create a random 4*4 matrix
[U,S,V] = svd(A);
A_p = cell(3,1); % rank-p approximation of A
for i = 1:3
    s = diag(S);
    A_p{i,1} = U(:,1:i) * diag(s(1:i)) * V(:,1:i)';
end
```

Then, I compute the Forbenius norm of the error in approximation.

```
% compute the error in approximation
err = zeros(3,1);
for i = 1:3
    err(i) = norm(A - A_p{i,1}, 'fro');
end
```

The approximation errors are [0.6679, 0.4221, 0.2132] as follows,



The image shows a MATLAB Variable Editor window titled 'Variable Editor - err'. It displays a 3x1 double array named 'err'. The values are 0.6679, 0.4221, and 0.2132 for indices 1, 2, and 3 respectively. The window also shows a toolbar with various icons and a status bar at the bottom.

	1	2	3	4	5	6	7	8
1	0.6679							
2	0.4221							
3	0.2132							
4								
5								
6								
7								
8								
9								
10								
11								
12								
13								
14								

4. (No need to turn this in). Run the Matlab snippet at this [link](#). Understand what is going on.

It's an application of SVD for low rank approximation in image compression.